Linear Regression

Plan

1. Linear regression
   - A geometrical interpretation
   - A probabilistic intepretation
2. Regularization
   - ℓ_2 regularization (Ridge regression)
   - ℓ_1 regularization (Lasso regression)

Linear Regression

- Similar to classification, we are given:
  - $\mathbf{X} : n \times d$ the training data matrix
  - $\mathbf{y} : n \times 1$ the target vector, where now $y_i \in R$
- and we want to find a linear model $\mathbf{w}$ such that:

$$y_i = \mathbf{w}^T \mathbf{x}_i$$

or equivalently:

$$\mathbf{y} = \mathbf{X}\mathbf{w}$$

- However most of the times there will not be an exact solution
- thus as in classification we need a measure of error

Linear Regression error function

- Probably the most well known and often used error function in learning

$$(y - z)^2, \text{ where } z = \mathbf{w}^T\mathbf{x}$$

  the *squared error*

- thus the error over the training set is given by:

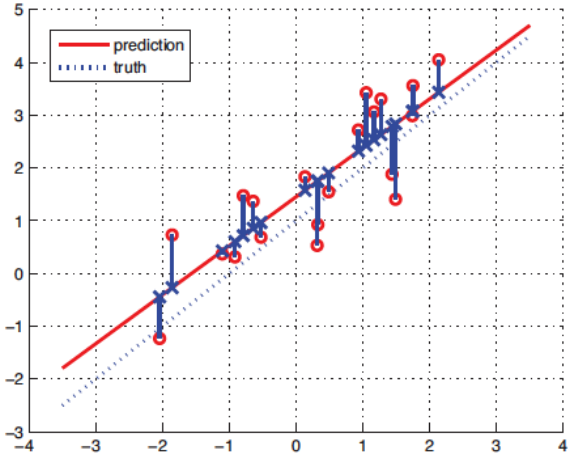
$$E(\mathbf{w}) = \sum (y_i - \mathbf{w}^T\mathbf{x}_i)^2 = ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

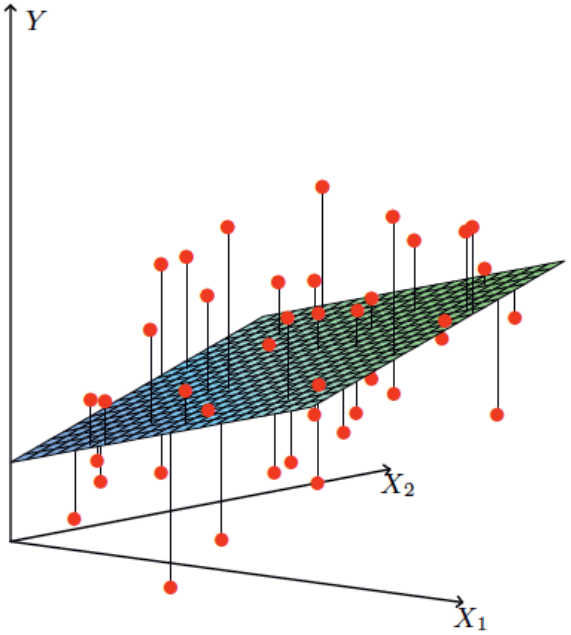
- and as before we seek to solve the optimization problem:

$$\min_{\mathbf{w}} ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 \tag{1}$$

  also known as the *minimization of the sum of squares*

Linear Regression error function





Figures taken from [3] and [1]

Minimizing the sum of squares

- Can be done by...

Minimizing the sum of squares

- Can be done by... deriving and setting to zero

$$
\begin{aligned}
\nabla E(\mathbf{w}) &= \nabla(\mathbf{y}-\mathbf{X}\mathbf{w})^T(\mathbf{y}-\mathbf{X}\mathbf{w}) \\
&= \nabla(\mathbf{y}^T\mathbf{y}-\mathbf{y}^T\mathbf{X}\mathbf{w}-\mathbf{w}^T\mathbf{X}^T\mathbf{y}+\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}) \\
&= \nabla(\mathbf{y}^T\mathbf{y}-\mathbf{y}^T\mathbf{X}\mathbf{w}-\mathbf{y}^T\mathbf{X}\mathbf{w}+\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}) \\
&= -2\mathbf{X}^T\mathbf{y}+(\mathbf{X}^T\mathbf{X}+(\mathbf{X}^T\mathbf{X})^T)\mathbf{w} \\
&= -2\mathbf{X}^T\mathbf{y}+2(\mathbf{X}^T\mathbf{X})\mathbf{w} \\
&= -2\mathbf{X}^T(\mathbf{y}-\mathbf{X}\mathbf{w})
\end{aligned}
$$

- Thus:

$$
\begin{aligned}
\nabla E(\mathbf{w}) = 0 \quad &\Rightarrow \quad \mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y} \\
&\Rightarrow \quad \text{assuming that } \mathbf{X}^T\mathbf{X} \text{ is invertible (non-singular)} \Rightarrow \\
&\Rightarrow \quad \mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}
\end{aligned}
$$

Comments on the solution

- What can we say about the gradient of the loss function ?

$$\nabla E(\mathbf{w}) \quad = \quad -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

Comments on the solution

- What can we say about the gradient of the loss function ?

$$\nabla E(\mathbf{w}) \quad = \quad -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

  It is a weighted sum of the learning instances where each instance contributes according to the error.
- Even though we have a closed form solution we can also do gradient descent, batch or on-line, in the later case the updates would be:

Comments on the solution

- What can we say about the gradient of the loss function ?

$$\nabla E(\mathbf{w}) \quad = \quad -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

  It is a weighted sum of the learning instances where each instance contributes according to the error.
- Even though we have a closed form solution we can also do gradient descent, batch or on-line, in the later case the updates would be:

$$\mathbf{x}(y - \mathbf{x}^T\mathbf{w})$$

- $\mathbf{X}^\dagger = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is a particular *left inverse* of $\mathbf{X}$ which we call *pseudoinverse*
- Why could it make sense to do gradient descent instead of getting the closed form solution?

Comments

- What can we say about the $(\mathbf{X}^T\mathbf{X})$ matrix?

Comments

- What can we say about the $(\mathbf{X}^T\mathbf{X})$ matrix? it is the covariance matrix if the data are centered (i.e. mean removed)
- If the columns of $\mathbf{X}$ are not linearly independent (when does that happen?) then:
  - $\mathbf{X}$ is not of full rank
  - $\mathbf{X}^T\mathbf{X}$ is singular
  - we do not have a unique solution $\mathbf{w}$
- The same rank deficiency problems we have when $d > n$
- In both cases above the solution is either to remove features or to use *model regularization*

Linear regression, a geometric interpretation, I

- Given the training matrix $\mathbf{X} : n \times d$ and the linear model $\mathbf{Xw}$
- The d *columns* ($n \times 1$ dimensional vectors; what the columns correspond to?) of the $\mathbf{X}$ matrix
  - act as the basis of a d-dimensional linear subspace (the column space of $\mathbf{X}$, noted by $C(\mathbf{X})$
  - which is embedeed in the R^n dimensional space
- We have:
  - $\mathbf{y} \in R^n$
  - $\hat{\mathbf{y}} = \mathbf{Xw} \in C(\mathbf{X})$. Why?
- In fact $\hat{\mathbf{y}}$ is the vector in $C(\mathbf{X})$ that has the minimum distance from $\mathbf{y}$, i.e. it is the projection of $\mathbf{y}$ on $C(\mathbf{X})$. Why?

Linear regression, a geometric interpretation, I

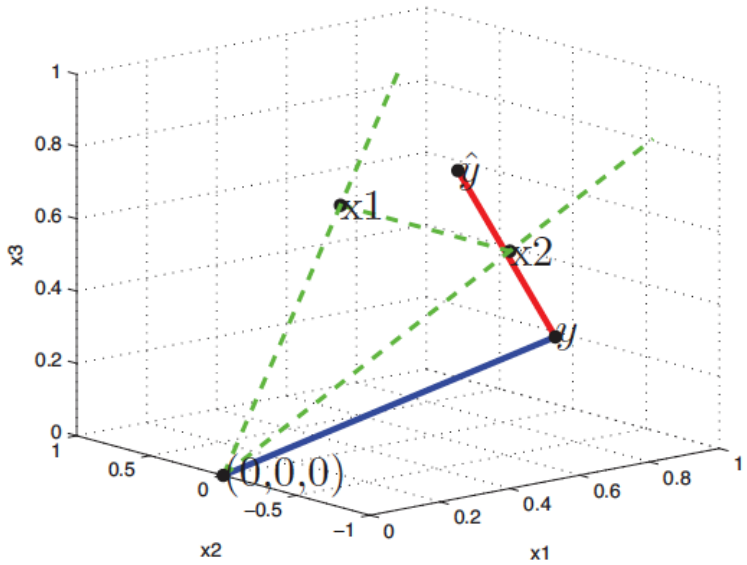
- Given the training matrix $\mathbf{X} : n \times d$ and the linear model $\mathbf{Xw}$
- The d *columns* ($n \times 1$ dimensional vectors; what the columns correspond to?) of the $\mathbf{X}$ matrix
  - act as the basis of a d-dimensional linear subspace (the column space of $\mathbf{X}$, noted by $C(\mathbf{X})$
  - which is embedeed in the R^n dimensional space
- We have:
  - $\mathbf{y} \in R^n$
  - $\hat{\mathbf{y}} = \mathbf{Xw} \in C(\mathbf{X})$. Why?
- In fact $\hat{\mathbf{y}}$ is the vector in $C(\mathbf{X})$ that has the minimum distance from $\mathbf{y}$, i.e. it is the projection of $\mathbf{y}$ on $C(\mathbf{X})$. Why? because it is the solution of:

$$\min_{\mathbf{w}} ||\mathbf{y} - \mathbf{Xw}||^2$$

- $\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$: the projection of $\mathbf{y}$ on $C(\mathbf{X})$

Linear regression, a geometric interpretation, II, [3]

- Let $\mathbf{X} = \begin{pmatrix} 1 & 2 \\ 1 & -2 \\ 1 & 2 \end{pmatrix} : n = 3, d = 2, \mathbf{y} = \begin{pmatrix} 8.8957 \\ 0.6130 \\ 1.7761 \end{pmatrix}$



- The two features x_1 and x_2 of $\mathbf{X}$ define the green two-dimensional hyperplane which is the $C(\mathbf{X})$ space
- $\mathbf{y} \in R^3, \hat{\mathbf{y}} \in C(\mathbf{X})$, $\hat{\mathbf{y}} - \mathbf{y}$ the residual vector for which $\min_{\mathbf{w}} ||\hat{\mathbf{y}} - \mathbf{y}||^2$

Linear regression, a parametric model

- We can also derive the linear regression optimization problem under some specific model assumptions
- We assume that the target value y is a linear function of $\mathbf{x}$ plus some noise ϵ, i.e.

$$y = \mathbf{w}^T\mathbf{x} + \epsilon$$

where $\epsilon \sim N(0, \sigma^2)$ or:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = N(y|\mu(\mathbf{x}), \sigma^2(\mathbf{x}))$$

- The simplest assumption is:

$$\mu(\mathbf{x}) = \mathbf{w}^T\mathbf{x}, \sigma^2(\mathbf{x}) = \sigma^2$$

- As usual we will assume that the training examples are independent and indentically distributed.

Linear regression, a parametric model: the error function, I

- We will use the...

Linear regression, a parametric model: the error function, I

- We will use the... data likelihood as the cost function
- which is given by...

Linear regression, a parametric model: the error function, I

- We will use the... data likelihood as the cost function
- which is given by...

$$
\begin{aligned}
E(\mathbf{w}) = p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) &= \prod_{i=1}^{i=n} p(y_i|\mathbf{x}_i, \boldsymbol{\theta}) = \prod_{i=1}^{i=n} N(y|\mu(\mathbf{x}), \sigma^2(\mathbf{x})) \\
&= \prod_{i=1}^{i=n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{1}{2\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i)^2)
\end{aligned}
$$

- and for convenience the negative log-likelihood:

$$
\begin{aligned}
-\ln(E(\mathbf{w})) &= -\sum_{i=1}^{N} \ln[\frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{1}{2\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i))] \\
&= -\sum_{i=1}^{N} \ln(\frac{1}{\sqrt{2\pi\sigma^2}}) + \sum_{i=1}^{N} \frac{1}{2\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i)^2
\end{aligned}
$$

Linear regression, a parametric model: the error function, II

- So minimizing the negative likelihood gives:

$$\begin{aligned}
\arg\min_{\mathbf{w}} -\ln(E(\mathbf{w})) &= \arg\min_{\mathbf{w}} -\sum_{i=1}^{N} \ln(\frac{1}{\sqrt{2\pi\sigma^2}}) \\
&+ \sum_{i=1}^{N} \frac{1}{2\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i)^2 \\
&= \arg\min_{\mathbf{w}} \sum_{i=1}^{N} (y_i - \mathbf{w}^T\mathbf{x}_i)^2
\end{aligned}$$

- which is a minimization of a sum of squares problem.

Rank deficiencies for $\mathbf{X}$ and regularization

- When we have redundant features the rank of $\mathbf{X}$ is lower than d and $\mathbf{X}^T\mathbf{X}$ is not invertible
- or when $d > n$ then the rank of $\mathbf{X}$ is at most n and as before $\mathbf{X}^T\mathbf{X}$ is not invertible
- In both cases the linear system

$$\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

  is underdetermined (more unknowns than independent equations) and does not have a unique solution.
- For example when we have two perfectly correlated attributes a large positive coefficient in one of them can be compensated by a large negative in the other. The w_i can have high variance, small changes in training set large changes in w_is.

Regularization, ℓ_2 norm, ridge regression, I

- We want to get a unique solution
- One way to do that is to control the size of the w_is by defining the constrained optimization problem:

$$\min_{\mathbf{w}} \quad \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + w_0))^2 \tag{2}$$

$$\text{subject to} \quad \sum_{j=1}^{d} w_i^2 = \ell_2^2(\mathbf{w}) \leq s$$

  note that now we do not include w_0 in $\mathbf{w}$, the norm is only applied on the coefficients not on the distance of the hyperplane form the origin
- The term $\sum_{j=1}^{d} w_i^2$ is a model complexity term called a regularizer.

Regularization, ℓ_2 norm, ridge regression, II

- The optimization problem 2 can be written as

Regularization, ℓ_2 norm, ridge regression, II

- The optimization problem 2 can be written as

$$\min_{\mathbf{w}} E_{\ell_2}(\mathbf{w}) \quad = \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} w_i^2 \quad (3)$$

  using the Langrange method, $\lambda \geq 0$. Optimization problems 2 and 3 are known as ridge regression.
- They have a closed form solution, found by deriving eq 3 and setting to zero, which is given by:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \quad (4)$$

  for ridge regression ([1], section 3.4.1).

Model complexity regularization

- The cost function of Ridge regression:

$$E_{\ell_2}(\mathbf{w}) = \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} w_i^2$$

is a trade off between an error component and a model complexity component:

$$\min_{\mathbf{w}} \qquad \text{Error}(\mathbf{w}) + \lambda \text{complexity}(\mathbf{w})$$

such trade-offs are used all the time in learning

Model complexity regularization

- A complex model can fit perfectly any training dataset but will be useless outside it.
- We need to find the right balance between training data fit (error) and model complexity

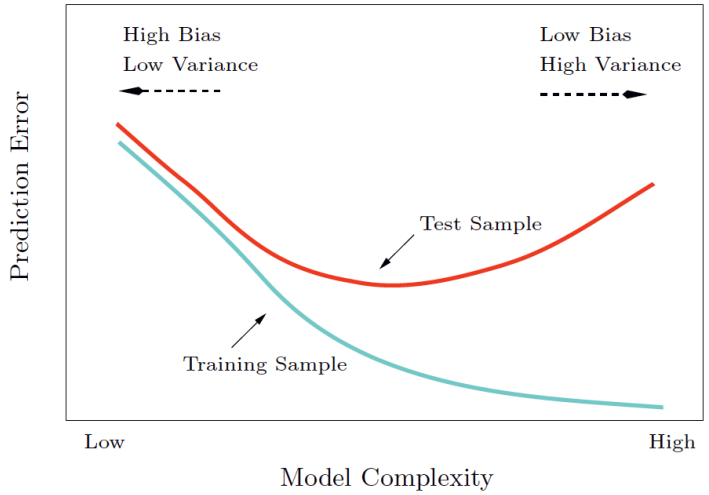


Figure: Training and testing as a function of model complexity, [1]

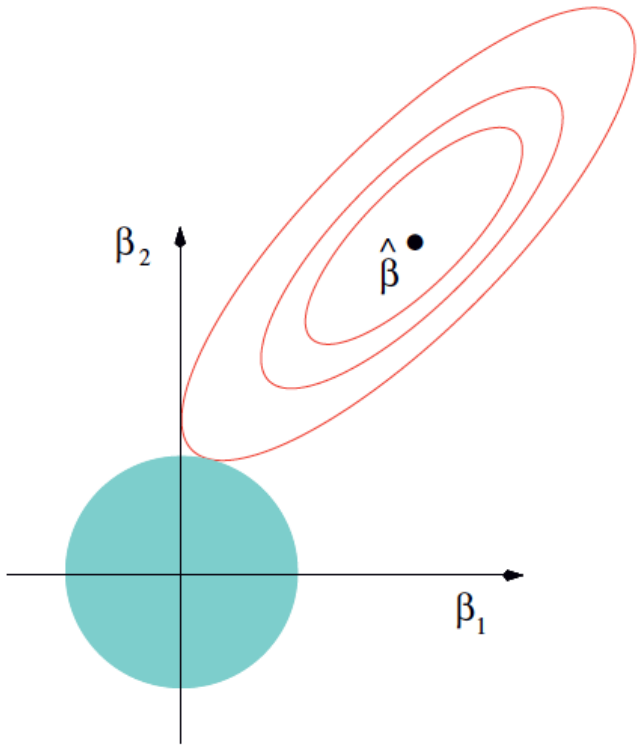
Ridge regression, comments, I

- The effect of:
  - the constraint $\sum_{j=1}^{d} w_i^2 \leq s$ as $s \to 0$
  - or equivalently of the term $\lambda \sum_{j=1}^{d} w_i^2$ as $\lambda \to \infty$
- is to shrink the coefficients towards zero, i.e. simplify the model.
- How does the Ridge optimization problem, eq 2, look like?

Ridge regression, comments, I

- The effect of:
  - the constraint $\sum_{j=1}^{d} w_i^2 \leq s$ as $s \to 0$
  - or equivalently of the term $\lambda \sum_{j=1}^{d} w_i^2$ as $\lambda \to \infty$
- is to shrink the coefficients towards zero, i.e. simplify the model.
- How does the Ridge optimization problem, eq 2, look like?

Ridge regression optimization problem visualization: In red the contours of the error function, sum of squares, in turquoise the model norm ball given by s ($\boldsymbol{\beta} := \mathbf{w}$).



Ridge regression, comments, II

- $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T$ recovers the pseudoinverse of $\mathbf{X}$ as $\lambda \to 0$.
- $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is always invertible, even if $\mathbf{X}^T\mathbf{X}$ is not, and was the original motivation of ridge regression
- Ridge solutions are not equivariant under scaling of the inputs so we need to standardize the inputs before solving 3
- If we had add w_0, the intercept, in the regularizer would make the procedure depend on the origin chosen for $\mathbf{y}$, i.e. adding a constant to each of the targets y_i would not simply result in a shift of the predictions by the same constant.

Other regularizers

- A regularization term controls the structure of the model and there are many ways to do that
- Given Ridge regression the most obvious next choise is...

Other regularizers

- A regularization term controls the structure of the model and there are many ways to do that
- Given Ridge regression the most obvious next choise is... $\ell_1(\mathbf{w})$ regularization:

$$\ell_1(\mathbf{w}) = \sum_{i=1}^{d} |w_i|$$

- The optimization problem

$$\begin{aligned} \min_{\mathbf{w}} \quad & \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + w_0))^2 \\ \text{subject to} \quad & \sum_{j=1}^{d} |w_i| = \ell_1(\mathbf{w}) \leq s \end{aligned} \tag{5}$$

Other regularizers

- or its equivalent form:

$$\min_{\mathbf{w}} E_{\ell_1}(\mathbf{x}) \quad = \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} |w_i| \quad (6)$$

  is known as the *lasso* regression.
- A main difference from ridge regression is that the ℓ_1 penalty will force many coefficients w_i to be exactly zero, thus performing feature selection.

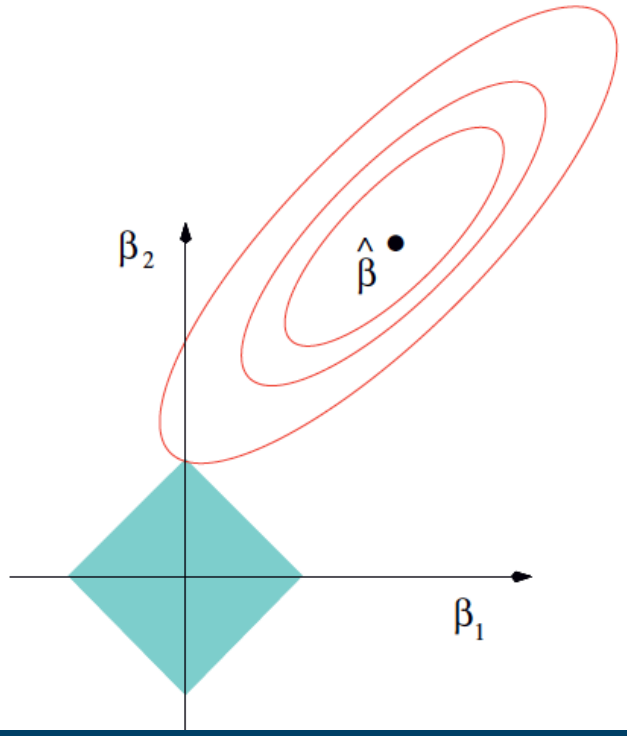
Lasso regression, comments

- The effect of:
  - the constraint $\sum_{j=1}^{d} |w_i| \leq s$ as $s \to 0$
  - or equivalently of the term $\lambda \sum_{j=1}^{d} |w_i|$ as $\lambda \to \infty$
- is to shrink more and more coefficients *exactly to* zero, i.e. essentially performing feature selection.
- How does the Lasso optimization problem, eq 2, look like?

Lasso regression, comments

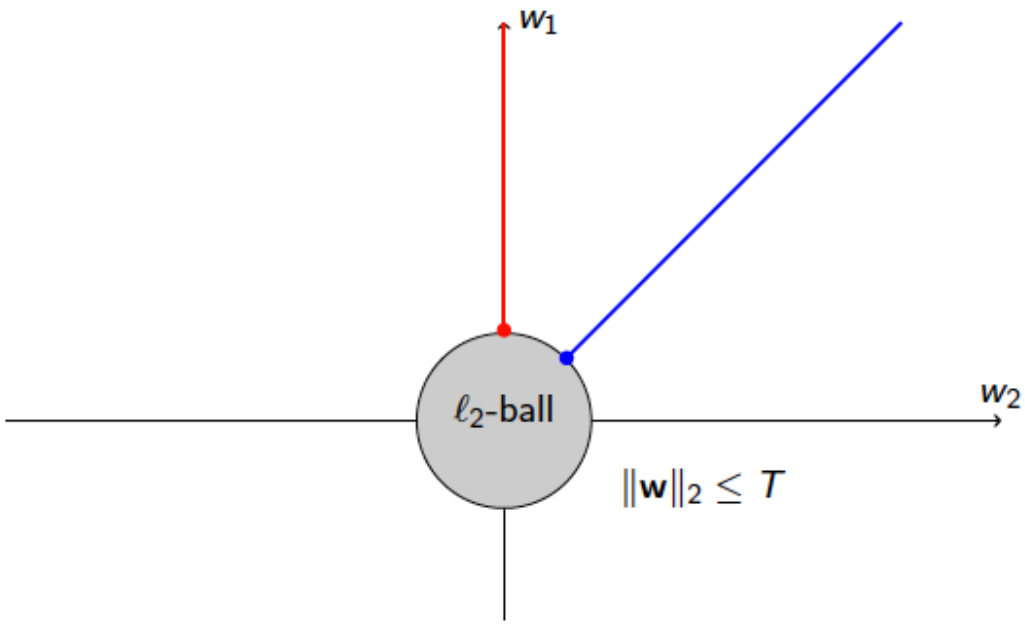
- The effect of:
  - the constraint $\sum_{j=1}^{d} |w_i| \leq s$ as $s \to 0$
  - or equivalently of the term $\lambda \sum_{j=1}^{d} |w_i|$ as $\lambda \to \infty$
- is to shrink more and more coefficients *exactly to* zero, i.e. essentially performing feature selection.
- How does the Lasso optimization problem, eq 2, look like?

Lasso regression optimization problem visualization: In red the contours of the error function, sum of squares, in turquoise the model norm ball given by s ($\boldsymbol{\beta} := \mathbf{w}$).



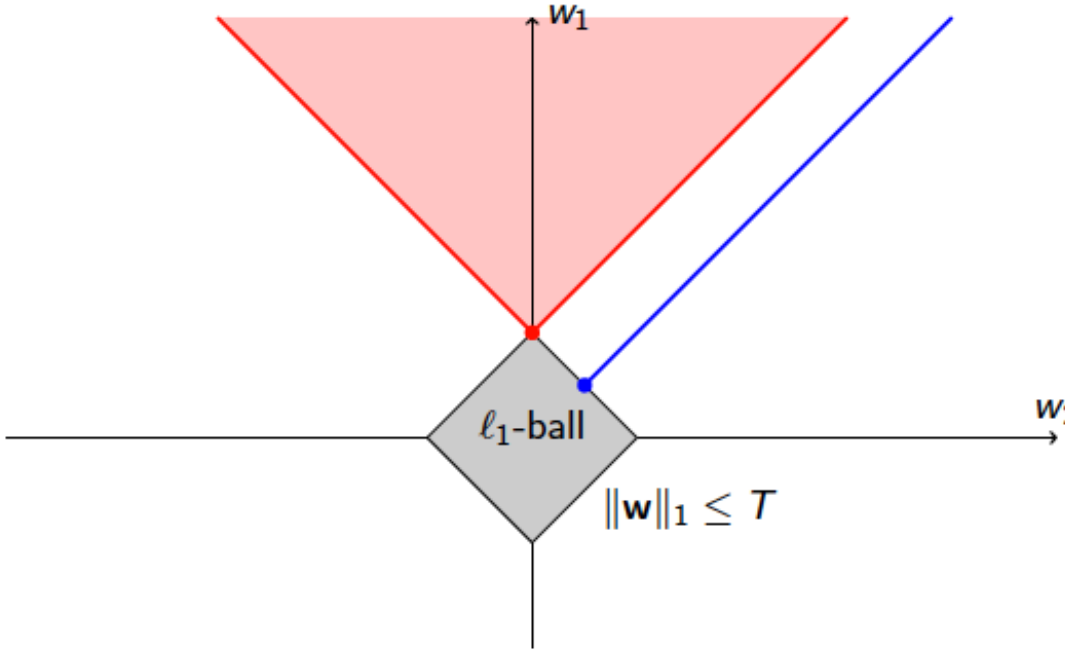
How Ridge and Lasso Actually Behave, I, [2]

Under Ridge the w_2 coefficient will be *exactly* zero only when the optimal solution lies on the w_1 axis



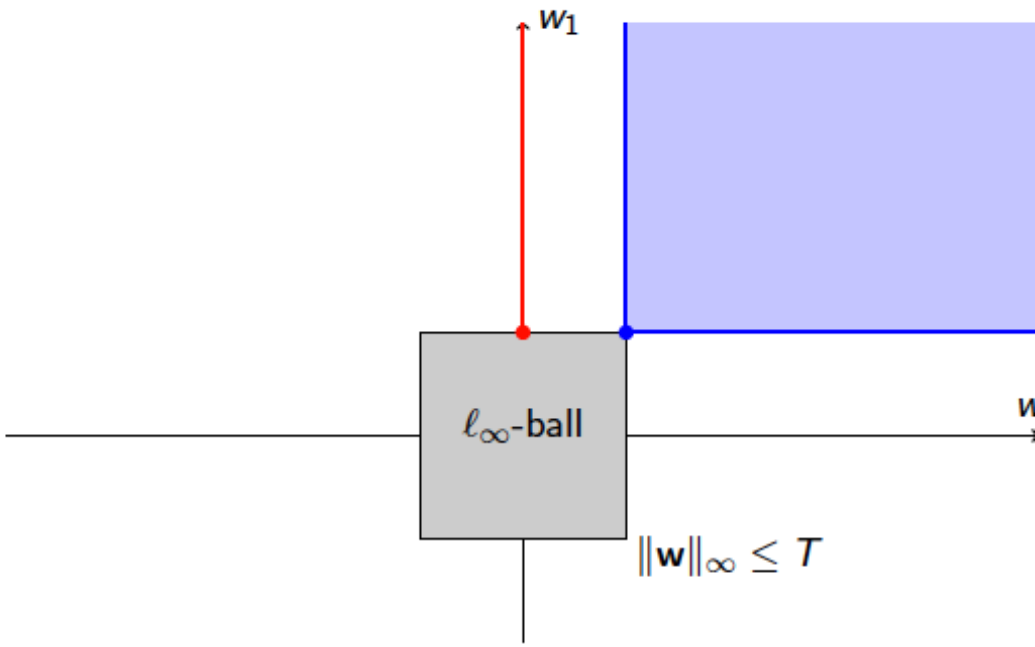
How Ridge and Lasso Actually Behave, II

Under Lasso the w_2 coefficient will be *exactly* zero when the optimal solution lies in the red area



How the ℓ_∞ Behaves

- $\ell_\infty(\mathbf{w}) = \max_{w_i}\{|w_i| | i = 1 \ldots d\}$
- It encourages $|w_1| = |w_2|$



How to solve the lasso optimization problem

- So how do we go about solving:

$$\min_{\mathbf{w}} \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} |w_i| \tag{7}$$

How to solve the lasso optimization problem

- So how do we go about solving:

$$\min_{\mathbf{w}} \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} |w_i| \tag{7}$$

- take the gradient and set to zero

How to solve the lasso optimization problem

- So how do we go about solving:

$$\min_{\mathbf{w}} \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} |w_i| \tag{7}$$

- take the gradient and set to zero does not work... Why?

How to solve the lasso optimization problem

- So how do we go about solving:

$$\min_{\mathbf{w}} \quad \sum_{i=1}^{N}(y_i - (\mathbf{w}^T\mathbf{x}_i + w_0))^2 + \lambda \sum_{j=1}^{d} |w_i| \tag{7}$$

- take the gradient and set to zero does not work... Why? $|x|$ is non-differentiable.
- Here comes in the subgradient

The subgradient

- The subgradient is the extension of gradient to *convex* but *non-differentiable* functions
- Given a *convex* function $f : R^p \rightarrow R$ and a vector $\mathbf{x}_i \in R^p$ a vector $\mathbf{z}$ is called *subgradient* at $\mathbf{x}_i$ if:

$$f(\mathbf{x}_i) + \mathbf{z}^T(\mathbf{x} - \mathbf{x}_i) \leq f(\mathbf{x}), \ \forall \mathbf{x} \in R^p$$

- Note that:

$$f(\mathbf{x}_i) + \mathbf{z}^T(\mathbf{x} - \mathbf{x}_i)$$

is the...

The subgradient

- The subgradient is the extension of gradient to *convex* but *non-differentiable* functions
- Given a *convex* function $f : R^p \rightarrow R$ and a vector $\mathbf{x}_i \in R^p$ a vector $\mathbf{z}$ is called *subgradient* at $\mathbf{x}_i$ if:

$$f(\mathbf{x}_i) + \mathbf{z}^T(\mathbf{x} - \mathbf{x}_i) \leq f(\mathbf{x}), \ \forall \mathbf{x} \in R^p$$

- Note that:

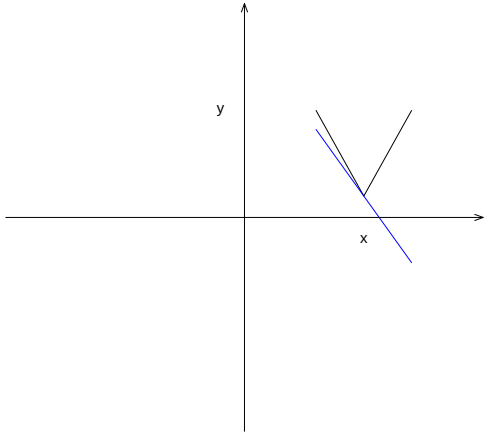
$$f(\mathbf{x}_i) + \mathbf{z}^T(\mathbf{x} - \mathbf{x}_i)$$

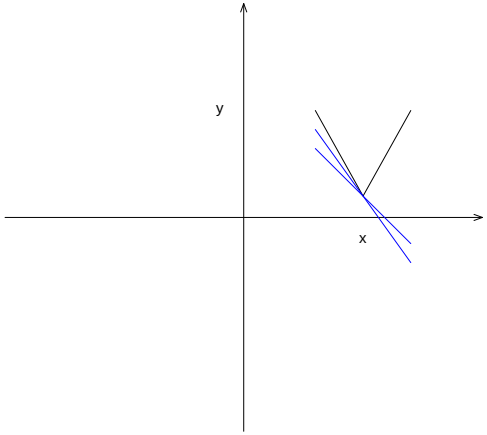
  is the... line that passes from $(\mathbf{x}_i, f(\mathbf{x}_i))$ and has a slope of $\mathbf{z}$
- remember that the tangent at $(\mathbf{x}_i, f(\mathbf{x}_i))$ is the line:

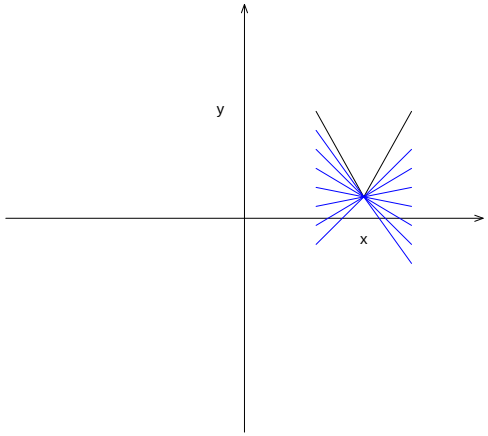
$$f(\mathbf{x}_i) + \nabla f(\mathbf{x}_i)^T(\mathbf{x} - \mathbf{x}_i)$$

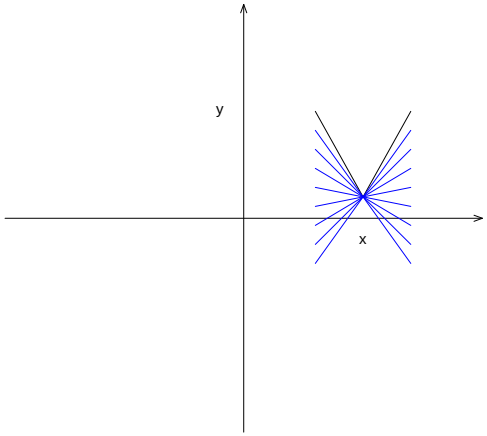
  for which if $f(\mathbf{x})$ is convex it holds that $\leq f(\mathbf{x})$.

Subgradients of $f(x) = \alpha|x - c| + b$ at $x = c$ (non-diff. point)









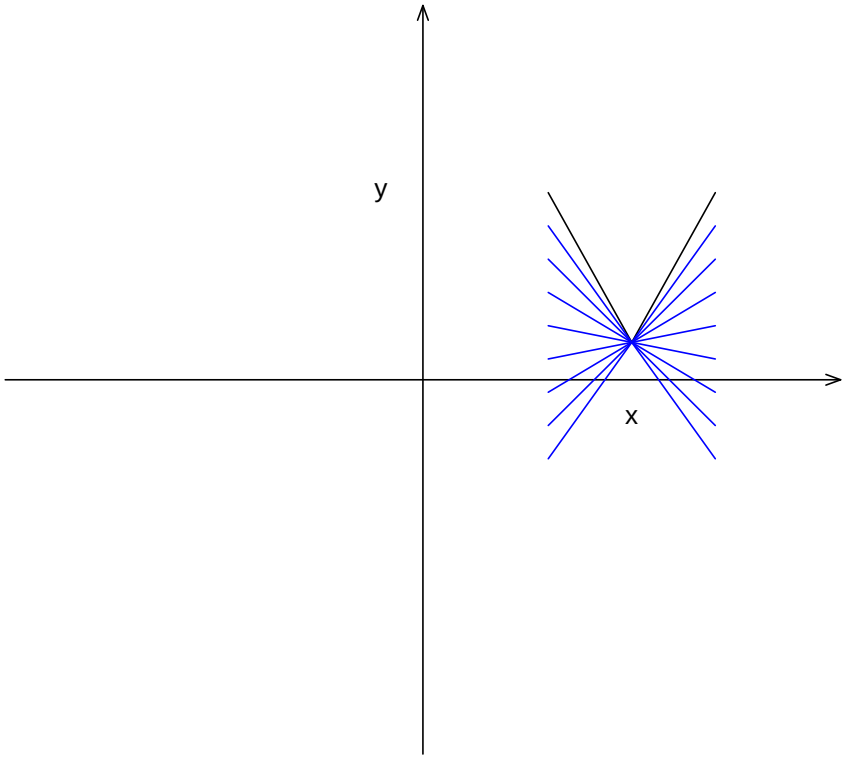
The subdifferential I

- Given a *convex* function $f : R^p \to R$ and a vector $\mathbf{x}_i \in R^p$ the *subdifferential* of f at $\mathbf{x}_i$ is defined as the *set*:

$$\partial f(\mathbf{x}_i) := \{\mathbf{z} \in R^p | f(\mathbf{x}_i) + \mathbf{z}^T(\mathbf{x} - \mathbf{x}_i) \leq f(\mathbf{x}),\ \forall \mathbf{x} \in R^p\}$$

  i.e. it is the set of all subgradient vectors $\mathbf{z}$
- Thus the subdifferential at a point $\mathbf{x}_i$ is a *set*
- so the subdifferential of a convex function at a point $\mathbf{x}_i$ is the set of all slope vectors $\mathbf{z}$ the lines of which pass from $(\mathbf{x}_i, f(\mathbf{x}_i))$ and are bellow the $f(\mathbf{x})$ for every value of $\mathbf{x}$

A subset of the subdifferential of $f(x) = \alpha|x - c| + b$ at $x = c$ (non-diff. point)



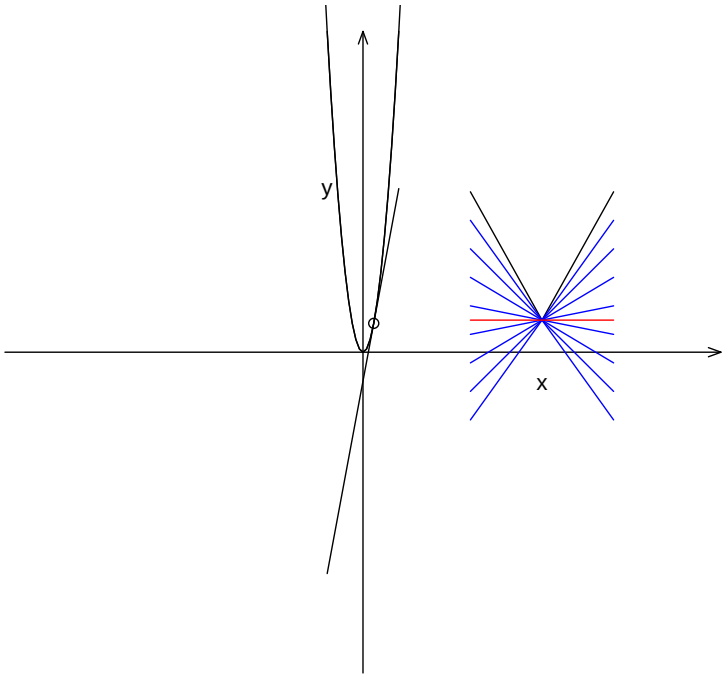
Subgradients at optimality I

- For any *convex* function $f : R^p \to R$, a point $\mathbf{x}_i \in R^p$ is a global minimum if and only if:

$$\mathbf{0} \in \partial f(\mathbf{x}_i)$$

  i.e. the $\mathbf{0}$ vector is in the subdifferential set.
- This is exactly the equivalent of $\nabla f(\mathbf{x}) = \mathbf{0}$ at the minimum of a *convex* function.

Subgradients at optimality II



Plots of $f(x) = x^2$ and its tangent at some point, and $g(x) = a|x - c| + b$ and its subgradients; 0 is in the subdifferential set $[-a, a]$ of $g(x)$ at $x = c$ and thus $g(c)$ is a global minimum.

Subgradient descent for lasso

- So instead of gradiend descent we now do subgradient descent
- And the update rule is:

$$w_{k+1} = w_k - \eta \partial E_{\ell_1}(\mathbf{w})$$

  where η is the learning rate.
- Not necessarily a descent method
  - in a descent method, every step is an improvement
- thus always keep track of the best $\mathbf{w}$ you have seen as we go

How does the Lasso Regression solution relates to the standard linear regression solution

- In the following slides we will see how the Lasso solution relates to the standard linear regression solution.
- We will do so by comparing their optimality conditions, i.e.

$$\nabla E(\mathbf{w}) = 0 \text{ and } \nabla E_{\ell_1}(\mathbf{w}) = 0$$

Lasso optimality conditions I

- The cost function of lasso regression can be written as:

$$E_{\ell_1}(\mathbf{w}) = E(\mathbf{w}) + \lambda \sum_{i=1}^{d} |w_i|$$

where $E(\mathbf{w})$ is the standard regression cost function.

- Remember:

$$\nabla E(\mathbf{w}) = \mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}$$

we will denote with:

$$[\nabla E(\mathbf{w})]_k = \frac{\partial E(\mathbf{w})}{\partial w_k}$$

the partial derivative of the standard regression cost function with respect to w_k

Lasso optimality conditions II

- Then the subdifferential of $E_{\ell_1}(\mathbf{w})$ with respect to w_i is given by:

$$\partial_{w_k} E_{\ell_1}(\mathbf{w}) = \frac{\partial E(\mathbf{w})}{\partial w_k} + \lambda \partial_{w_k} |w_k|$$

where:

$$\begin{aligned} \partial_{w_k} |w_k| &= \{-1\} \text{ if } w_k < 0 \\ \partial_{w_k} |w_k| &= [-1, 1] \text{ if } w_k = 0 \\ \partial_{w_k} |w_k| &= \{+1\} \text{ if } w_k > 0 \end{aligned}$$

Lasso optimality conditions III

- Thus:

$$\partial_{w_k} E_{\ell_1}(\mathbf{w}) = \frac{\partial E(\mathbf{w})}{\partial w_k} + \begin{cases} \{-\lambda\} & \text{if } w_k < 0 \\ [-\lambda, \lambda] & \text{if } w_k = 0 \\ \{\lambda\} & \text{if } w_k > 0 \end{cases}$$

and by requiring

$$\mathbf{0} \in \partial_{w_k} E_{\ell_1}(\mathbf{w})$$

we have:

$$\frac{\partial E(\mathbf{w})}{\partial w_k} \in \begin{cases} \{\lambda\} & \text{if } w_k < 0 \\ [-\lambda, \lambda] & \text{if } w_k = 0 \\ \{-\lambda\} & \text{if } w_k > 0 \end{cases} \tag{8}$$

so we need to solve for w_k for the three subcases of eq 8

Lasso optimality conditions IV

- One can show ([3], 13.3)

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = \alpha_k w_k - c_k \tag{9}$$

where:

$$\begin{aligned} \alpha_k &= 2\sum_{i=1}^{n} x_{ik}^2 \\ c_k &= 2\sum_{i=1}^{n} x_{ik}(y_i - \mathbf{w}_{-k}^T \mathbf{x}_{i,-k}) \end{aligned}$$

- c_k is proportional to the correlation of the kth feature to the residual of the model without the kth feature
- *Optimality conditions standard regression*:

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = 0 \Rightarrow w_k = c_k / a_k$$

Lasso optimality conditions V

- Thus from eq 8 and eq 9 we have:

$$
\begin{aligned}
\alpha_k w_k - c_k &\in \begin{cases} \{\lambda\} & \text{if } w_k < 0 \\ [-\lambda, \lambda] & \text{if } w_k = 0 \\ \{-\lambda\} & \text{if } w_k > 0 \end{cases} \\
&\Rightarrow \\
w_k &= \frac{c_k}{\alpha_k} + \frac{\lambda}{\alpha_k} (\text{ if } w_k < 0 \Rightarrow) \text{ when } c_k < -\lambda \\
w_k &= 0 \text{ when } c_k \in [-\lambda, \lambda] \\
w_k &= \frac{c_k}{\alpha_k} - \frac{\lambda}{\alpha_k} (\text{ if } w_k > 0 \Rightarrow) \text{ when } c_k > \lambda
\end{aligned}
$$

Soft thresholding I

- $\mathbf{0} \in \partial_{w_k} E_{\ell_1}(\mathbf{w})$ when:

$$w_k = \frac{c_k}{\alpha_k} + \frac{\lambda}{\alpha_k}(\text{ if } w_k < 0 \Rightarrow) \text{ when } c_k < -\lambda \quad (10)$$

$$w_k = 0 \text{ when } c_k \in [-\lambda, \lambda] \quad (11)$$

$$w_k = \frac{c_k}{\alpha_k} - \frac{\lambda}{\alpha_k}(\text{ if } w_k > 0 \Rightarrow) \text{ when } c_k > \lambda \quad (12)$$

- let:
  - $x_+ = \max(0, x)$ the postitive part of x
  - soft$(\alpha, \delta) = sign(\alpha)(|\alpha| - \delta)_+$: the *soft thresholding* function which shrinks the size of α by δ and if $\alpha < \delta$ sets it to zero.
- then we can rewrite eqs 10,11,12, as:

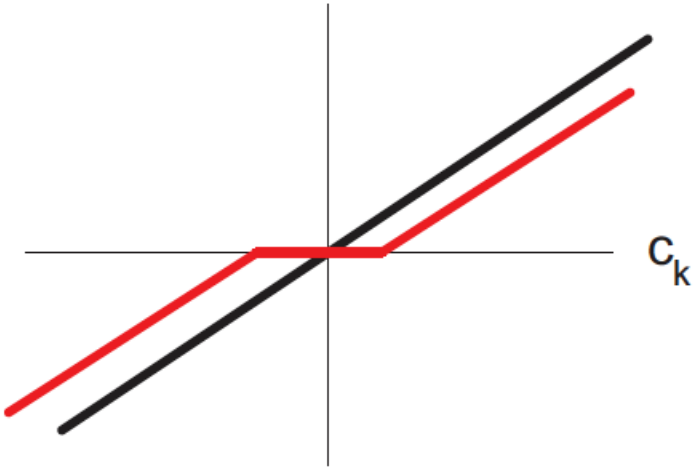
$$w_k = \text{soft}(\frac{c_k}{\alpha_k}, \frac{\lambda}{\alpha_k})$$

Soft thresholding vs hard thresholding I

Plots of w_k vs c_k

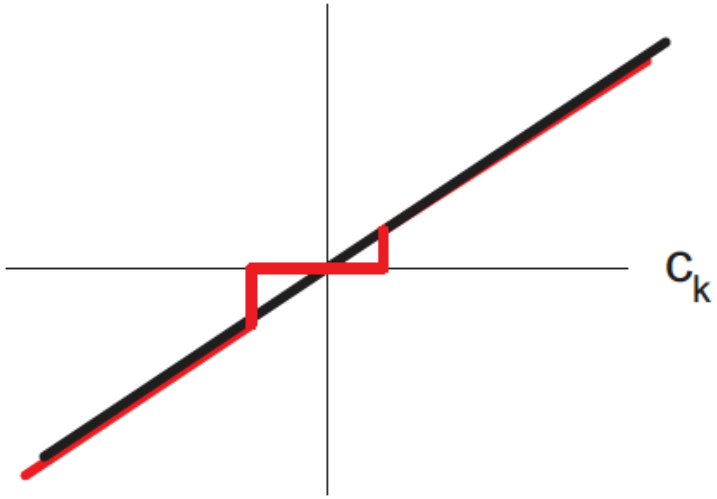
Soft Thresholding

$w_k = \text{soft}(\frac{c_k}{\alpha_k}, \frac{\lambda}{\alpha_k})$



Hard Thresholding

$w_k = 0$ if $c_k \in [-\lambda, \lambda]$



- Black lines: $w_k = c_k/a_k$ solution of standard regression
- Red lines: their sof/hard thresholed variants
- set to zero features with $c_k \in [-\lambda, \lambda]$
- Lasso stands for *least absolute selection and shrinkage operator*

Proximal gradient descent

- Used to optimize objectives of the form:

$$\min_{\mathbf{w} \in R^p} f(\mathbf{w}) + \lambda \Omega(\mathbf{w}) \tag{13}$$

- $f : R^p \rightarrow R$ is a *convex* and *differentiable* function or more formally a generic smooth function with Lipschitz-continuous gradient, i.e.

$$\frac{||f(\mathbf{w}_1) - f(\mathbf{w}_2)||}{||\mathbf{w}_1 - \mathbf{w}_2||} \leq L, \ \forall \mathbf{w}_1, \mathbf{w}_2 \in R^p$$

- $\Omega(\mathbf{w})$ is a *convex* function, often *non-smooth*, typically a sparsity inducing norm, e.g. $\ell_1(\mathbf{w})$
- Quite appropriate method for *large, non-smooth, convex* problems.

Intuition proximal gradient descent

1. First establish a solution vecort $\mathbf{w}_1$ by gradient descent *only* on $f(\mathbf{w})$, i.e. ignore $\Omega(\mathbf{w})$.
2. Get a new $\mathbf{w}_2$ that is close to $\mathbf{w}_1$ and also minimizes $\Omega(\mathbf{w})$
3. Typically $\mathbf{w}_2$ is found through the *proximal operator* of $\Omega(\mathbf{w})$ that often has a *closed form* solution.
4. Iterate 1-3 until convergence

It turns out that the proximal operator of:

$$\Omega(w) = \ell_1(\mathbf{w}) = \sum_i |w_i|$$

denoted by $\mathbf{prox}_{\lambda\ell_1(\mathbf{w})}(\mathbf{x})$ is given by:

$$[\mathbf{prox}_{\lambda\ell_1(\mathbf{w})}(\mathbf{w})]_i = \text{soft}(w_i, \lambda) \quad = \quad \begin{cases} w_i + \lambda & \text{if } w_i < -\lambda \\ 0 & -\lambda < w_i < \lambda \\ w_i - \lambda & \text{if } \lambda < w_i \end{cases}$$

Probabilistic Interpretations

- Both Ridge and Lasso have a probabilistic interpretation
- The solution of the standard linear regression is a maximum likelihood solution
- the solutions of Ridge and Lasso are Maximum a-posteriori estimates with specific priors on the parameters, i.e

$$\arg\max_{\mathbf{w}} p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \lambda) \propto p(\mathbf{y}|\mathbf{X}, \mathbf{w}) P(\mathbf{w}|\lambda)$$

and by taking the negative $\ln$ we have:

$$E_{\ell_2|\ell_1}(\mathbf{w}) = -\ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}) - \ln P(\mathbf{w}|\lambda) \tag{14}$$

$p(\mathbf{y}|\mathbf{X}, \mathbf{w})$ is the likelihood function of standard linear regression

Ridge regression prior and MAP

- The Ridge parameters prior is the Normal prior:

$$p(\mathbf{w}|\lambda) = \prod_{j=1}^{d} N(w_j|0, \lambda^2)$$

- and plugging that prior in eq 14 we get :

$$\begin{aligned} E_{\ell_2}(\mathbf{w}) &= \sum_{i=1}^{n} \frac{1}{2\sigma^2}(y_i - (w_0 + \mathbf{w}^T\mathbf{x}_i))^2 + \frac{1}{\lambda^2}||\mathbf{w}||_2^2 \\ &\propto E(\mathbf{w}) + \lambda'||\mathbf{w}||_2 \end{aligned}$$

$E(\mathbf{w})$ is the sum of squares cost function of linear regression, $\lambda' = \frac{2\sigma^2}{\lambda^2}$.

Lasso regression prior and MAP

- The lasso parameters prior is the Laplace prior:

$$p(\mathbf{w}|\lambda) = \prod_{j=1}^{d} \mathsf{Lap}(w_j|0, 1/\lambda) \propto \prod_{j=1}^{d} \exp(-\lambda|w_j|)$$

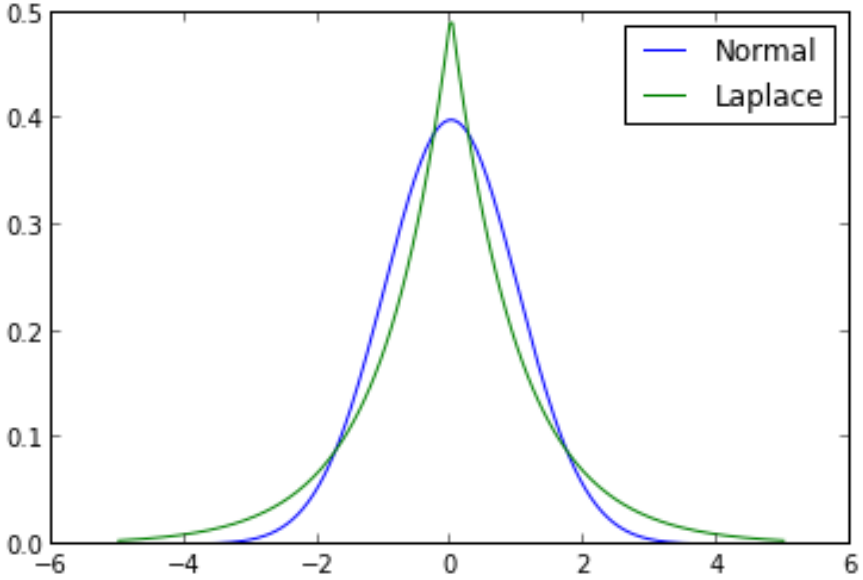
- and plugging that prior in eq 14 we get :

$$\begin{aligned} E_{\ell_1}(\mathbf{w}) &= \sum_{i=1}^{n} \frac{1}{2\sigma^2}(y_i - (w_0 + \mathbf{w}^T\mathbf{x}_i))^2 + \lambda||\mathbf{w}||_1 \\ &\propto E(\mathbf{w}) + \lambda'||\mathbf{w}||_1 \end{aligned}$$

$E(\mathbf{w})$ is the sum of squares cost function of linear regression, $\lambda' = 2\lambda\sigma^2$.

The Laplace distribution

- Lap$(x|\mu, b) := \frac{1}{2b}\exp(-\frac{|x-\mu|}{b})$, μ is a location parameter and $b > 0$ a scale parameter.
- mean $= \mu$, var $= 2b^2$
- Plot of $N(0,1)$, Lap$(0, 1/\sqrt(2)$, $var = 1$ for both.



Bibliography I

Trevor Hastie, Robert Tibshirani, and Jerome Friedman.
Elements of Statistical Learning.
Springer, second edition, 2008.

Julien Mairal.
Supervised feature selection in graphs with path coding penalties and network flows.
2012.
Talk at Neyman Seminar, UC Berkley.

Kevin Murphy.
Machine Learning: A probabilistic Perspective.
MIT, 2012.